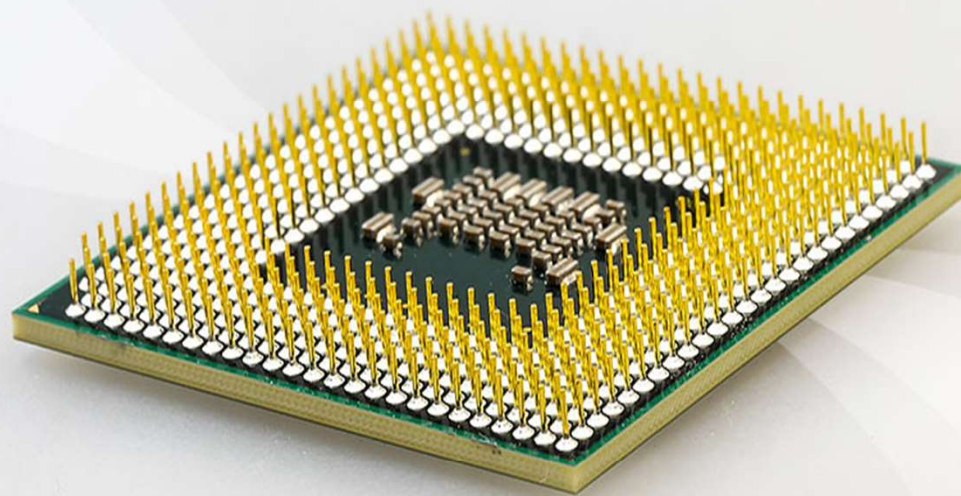


Đại Học Công Nghiệp TP HCM



THIẾT KẾ MẠCH TÍCH HỢP VỚI HDL

GV: Vũ Thị Hồng Nga

Biên soạn năm 2019

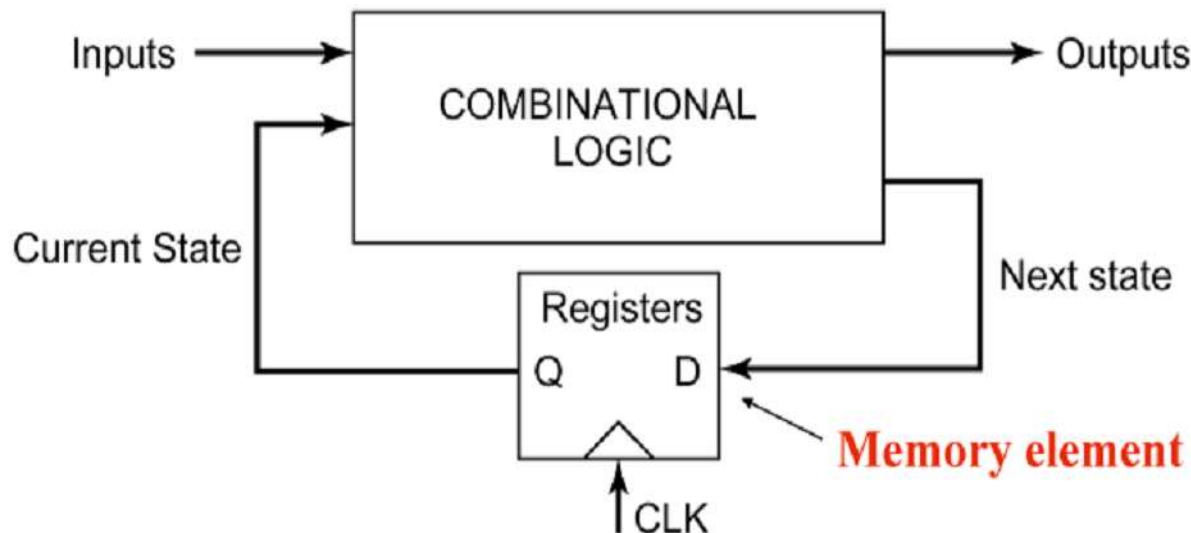
Chương 5: Thiết kế mạch tuần tự với Verilog HDL

Nội dung:

Mô tả các mạch tuần tự sau

- Flip Flop
- Mạch đếm nhị phân
- Mạch đếm BCD
- Máy trạng thái
- ROM/RAM
- Các mạch số ứng dụng

1. Đặc Điểm Chung Mạch Tuần Tự



- Mạch tuần tự là mạch logic có tính chất nhớ, có khâu trễ
- Trạng thái tiếp theo của mạch phụ thuộc vào giá trị của kích thích ở ngõ vào và trạng thái hiện tại của chính nó.
- Mạch tuần tự thường hoạt động đồng bộ theo sự điều khiển của tín hiệu xung nhịp clock



Các dạng thường gặp

ck tác động theo cạnh lên, rs không đồng bộ và tích cực thấp

```
always @ (posedge ck or negedge rs)
```

```
begin
```

```
if (rs == 1'b0)
```

```
...
```

```
end
```

ck tác động theo cạnh lên, rs không đồng bộ và tích cực cao

```
always @ (posedge ck or posedge rs)
```

```
begin
```

```
if (rs == 1'b1)
```

```
...
```

```
end
```



ck tác động theo cạnh lên, rs đồng bộ và tích cực thấp

```
always @ (posedge ck)
```

```
begin
```

```
if (rs == 1'b0)
```

```
...
```

```
end
```

ck tác động theo cạnh lên, rs đồng bộ và tích cực cao

```
always @ (posedge ck)
```

```
begin
```

```
if (rs == 1'b1)
```

```
...
```

```
end
```



ck tác động theo cạnh xuống, rs không đồng bộ và tích cực thấp

always @ (negedge ck or negedge rs)

begin

if (rs == 1'b0)

...

end

ck tác động theo cạnh xuống, rs không đồng bộ và tích cực cao

always @ (negedge ck or posedge rs)

begin

if (rs == 1'b1)

...

end



ck tác động theo cạnh xuống, rs đồng bộ và tích cực thấp

```
always @ (negedge ck)
begin
if (rs == 1'b0)
...
end
```

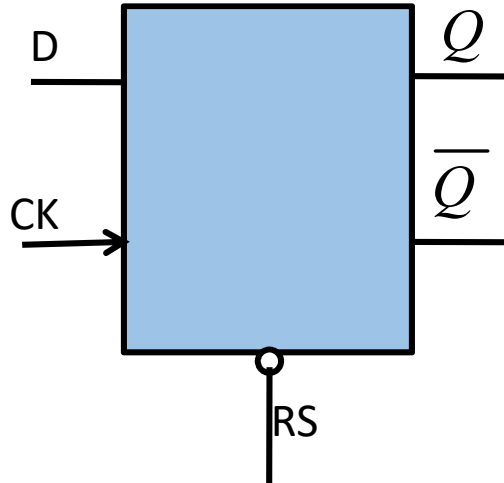
ck tác động theo cạnh xuống, rs đồng bộ và tích cực cao

```
always @ (negedge ck)
begin
if (rs == 1'b1)
...
end
```



2. Mô tả Flip Flop

Vd 1: mô tả D FF, ck tác động cạnh lên, **rs không đồng bộ tích cực thấp**



D _n	Q _{n+1}
0	0
1	1

```
module d_ff (ck, d, rs, q);  
  input ck, d, rs;  
  output q;  
  reg q;  
  always @ (posedge ck or negedge rs)  
  begin  
    if (rs == 1'b0)  
      q <= 0;  
    else  
      q <= d;  
  end  
endmodule
```



VD2: viết chương trình mô tả D_FF có ck tác động theo cạnh xuống, chân rs đồng bộ tích cực mức cao

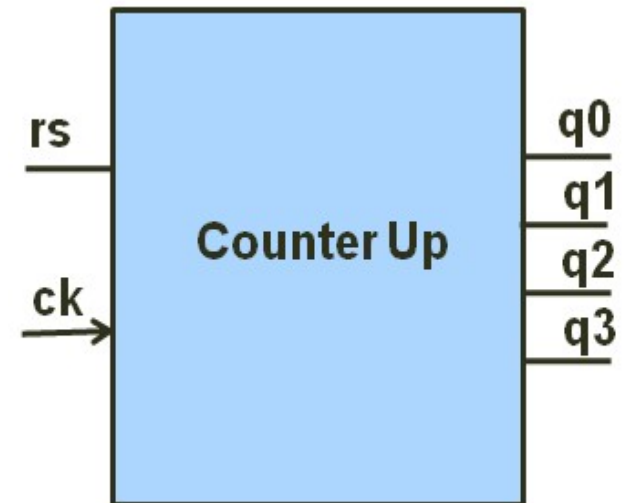
```
module d_ff (ck, d, rs, q);  
input ck, d, rs;  
output q;  
reg q;  
always @ (negedge ck )  
begin  
    if (rs == 1'b1)  
        q <= 0;  
    else  
        q <= d;  
    end  
end  
endmodule
```



3. Mô tả bộ đếm nhị phân

Ví dụ : mô tả bộ đếm lên nhị phân 4 bit, ck tác động theo cạnh lên, rs không đồng bộ tích cực mức cao

```
module counter_4bit (ck,rs,q);  
input ck, rs;  
output [3:0]q;  
reg [3:0]q;  
always @ (posedge ck or posedge rs)  
begin  
    if (rs == 1)  
        q <= 4'b 0000;  
    else q <= q + 1;  
end  
endmodule
```



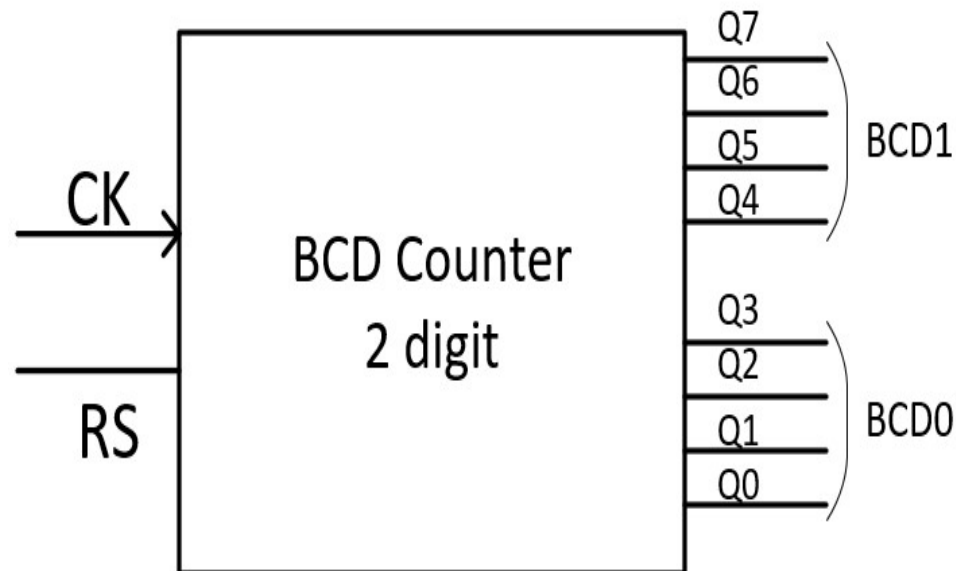
4. Mô tả bộ đếm BCD

VD 1 : viết chương trình verilog mô tả bộ đếm lên BCD 1 digit, chân rs tích cực mức cao, ck tác động theo cạnh lên.

Gợi ý: viết giống bộ đếm lên nhị phân từ 0 đến 9



VD2 : viết chương trình verilog mô tả bộ đếm lên BCD 2 digit, đếm lên từ 00 $\rightarrow$ 99, chân rs tích cực mức cao, ck tác động theo cạnh lên.



VD: mô tả mạch đếm 2 digit đếm 00 → 99

```
//mạch đếm 00-99
//ngo ra ma BCD
//bcd1 : hàng đơn vị
//bcd2 : hàng chục
//-----//
module counter_bcd (ck,rs,bcd1,bcd2);
input ck, rs;
output [3:0] bcd1,bcd0;
reg [3:0] bcd1, bcd0;
```

```
always @ (posedge ck)
begin
    if (rs == 1)
    begin
        bcd1 <= 4'b0000;
        bcd0 <= 4'b0000;
    end
    else
    begin
        bcd0 <= bcd0 + 1;
        if (bcd0 == 4'b1001)
        begin
            bcd0 <= 4'b0000;
            bcd1 <= bcd1 + 1;
            if (bcd1 == 4'b1001)
            bcd1 <= 4'b0000;
        end
    end
end
endmodule
```



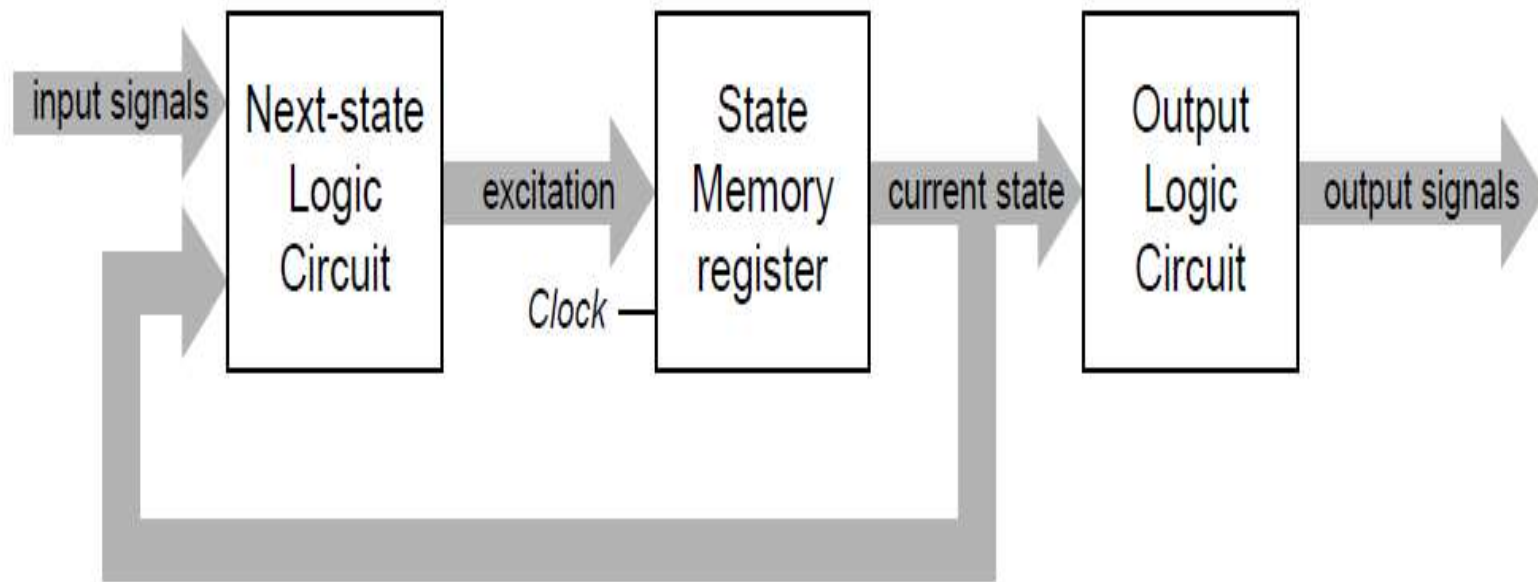
5. MÁY TRẠNG THÁI (Finite state machines -FSM)

a. Giới thiệu chung về FSM:

- FSM là một công cụ dùng để mô tả các hành vi của hệ thống số. Nó chỉ ra tất cả các trạng thái có thể có của hệ thống, và những sự kiện nào sẽ gây ra sự thay đổi trạng thái.
- Các ngõ ra của FSM phụ thuộc vào ngõ vào, xung ck và trạng thái nhớ của các flip flop bên trong FSM
- FSM thường được mô tả bằng giản đồ Moore hoặc Mealy



b. Sơ đồ khối FSM dạng moore



Next state logic circuit: khối tác động trạng thái kế tiếp

State memory register: khối thanh ghi (lưu trạng thái trong của FSM)

Output logic circuit: khối tác động ngõ ra



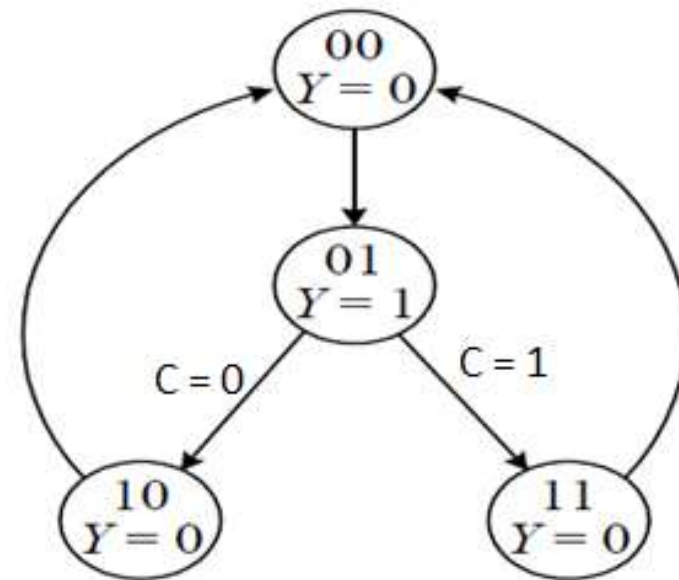
VD: FSM mô tả theo giản đồ trạng thái dạng Moore

Ngõ vào: C

Ngõ ra : Y

4 trạng thái trong: 00,01,10,11

Chiều mũi tên là chiều chuyển
trạng thái



Ta có thể mã hóa các trạng thái của FSM thành các nhãn mong muốn:

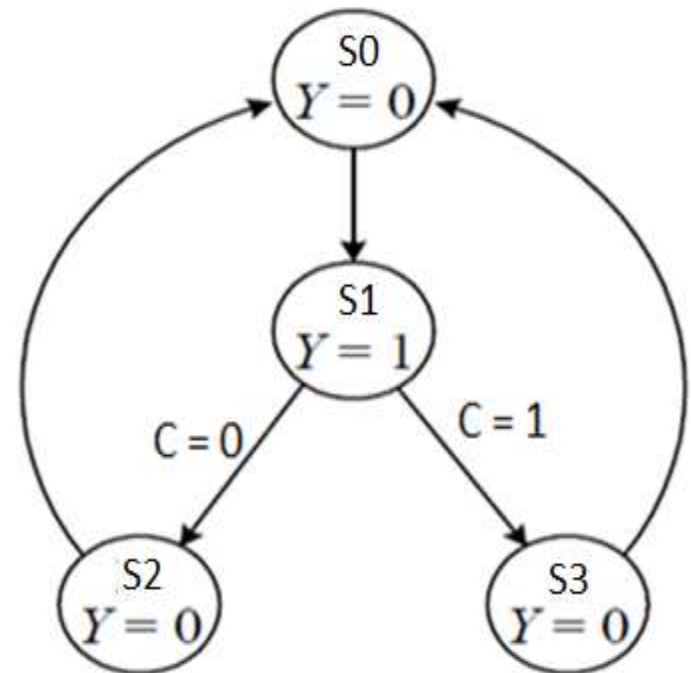
VD:

00 là s0

01 là s1

10 là s2

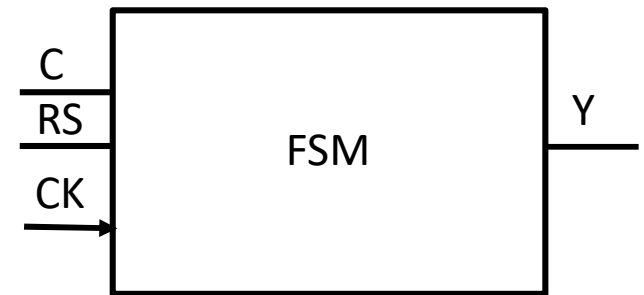
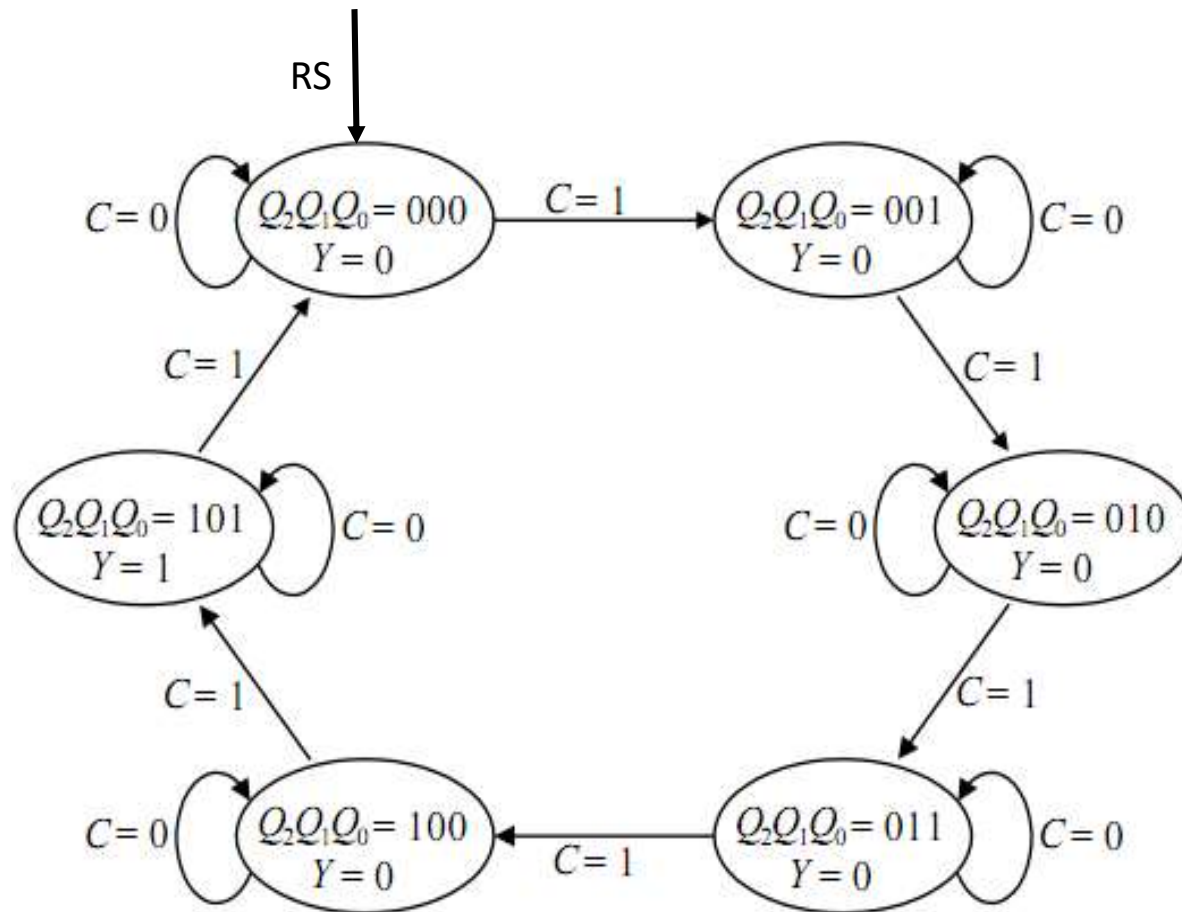
11 là s3



Ví dụ 1: Viết code Verilog mô tả máy trạng thái sau

Gráp trạng thái của FSM

Sơ đồ chân của FSM

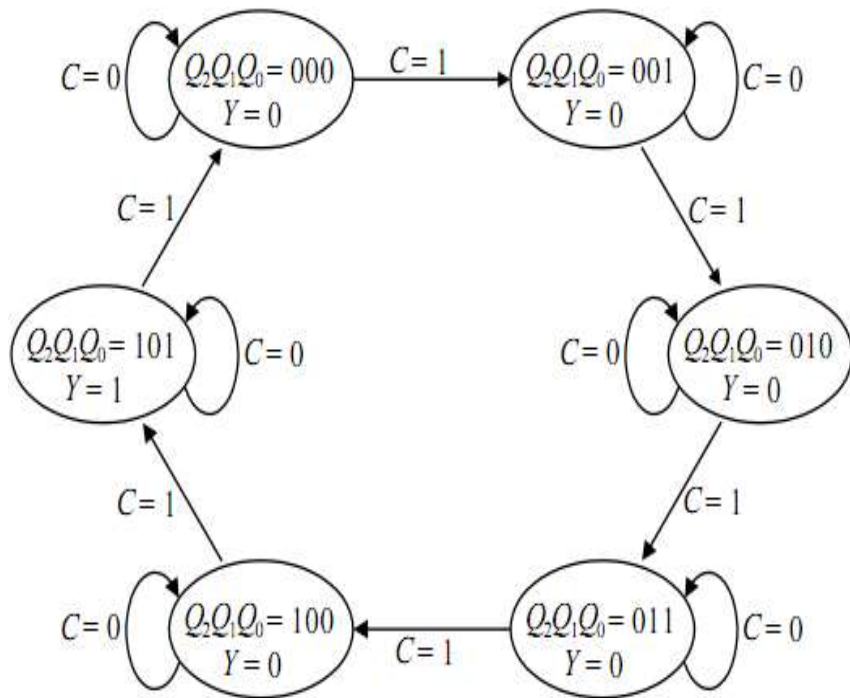


```
module fsm_VD1 (ck,rs,c,y);  
parameter s0 = 3'b000;  
parameter s1 = 3'b001;  
parameter s2 = 3'b010;  
parameter s3 = 3'b011;  
parameter s4 = 3'b100;  
parameter s5 = 3'b101;  
input ck,rs,c;  
output y;  
reg [2:0]current_state, next_state;  
reg y;
```

```
always @ (*)  
begin  
case (current_state)  
s0 : if (c) next_state = s1;  
     else next_state = current_state;  
s1 : if (c) next_state = s2;  
     else next_state = current_state;  
s2 : if (c) next_state = s3;  
     else next_state = current_state;  
s3 :   if (c) next_state = s4;  
       else next_state = current_state;  
s4 :   if (c) next_state = s5;  
       else next_state = current_state;  
s5 :   if (c) next_state = s0;  
       else next_state = current_state;  
default next_state = s0;  
endcase  
end
```



THIẾT KẾ MẠCH TÍCH HỢP VỚI HDL



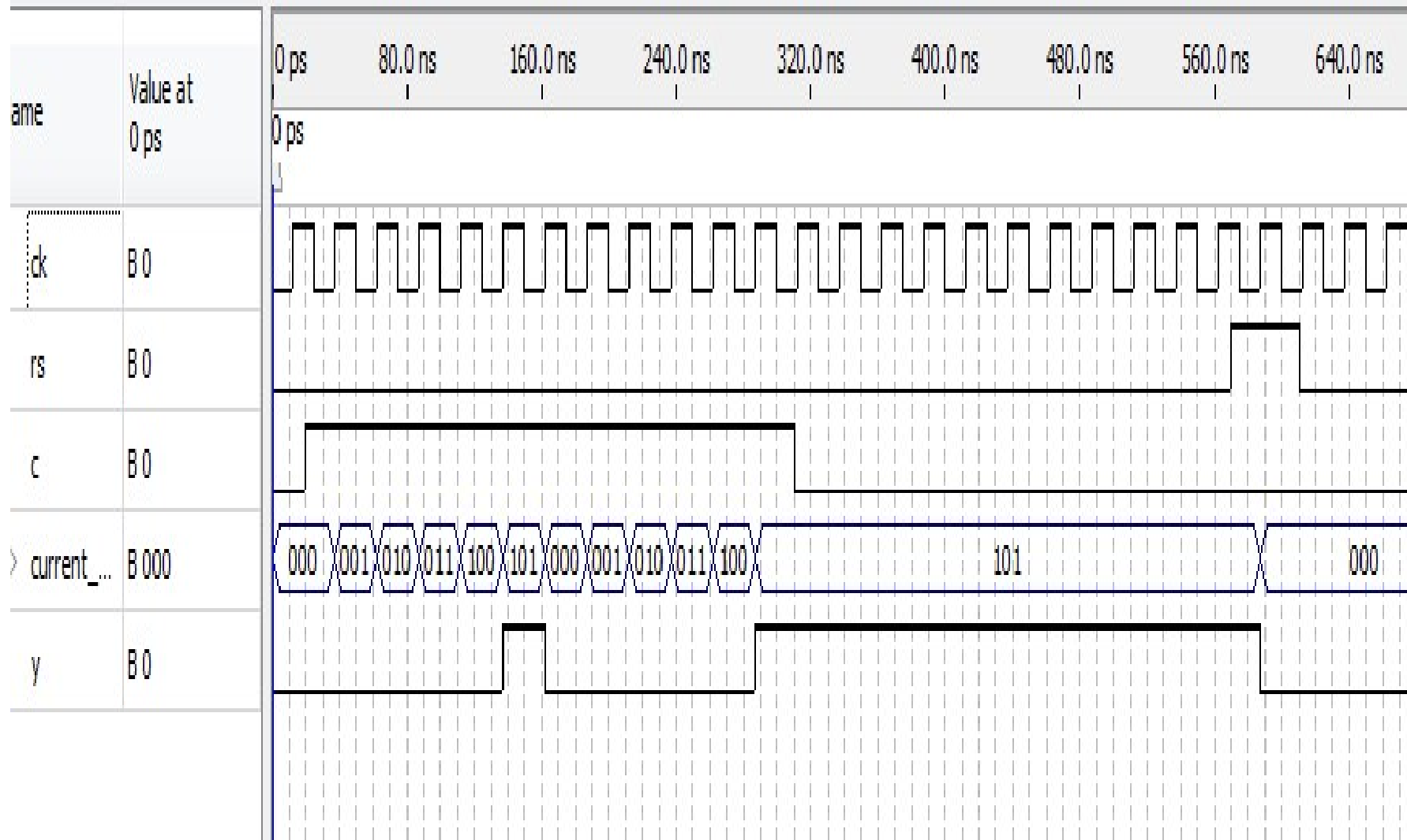
```

always @ (posedge ck)
begin
  if (rs == 1) current_state <= s0;
  else current_state <= next_state;
End
  
```

```

always @ (*)
begin
  if (current_state == s5) y = 1'b1;
  else y = 1'b0;
end
Endmodule
  
```

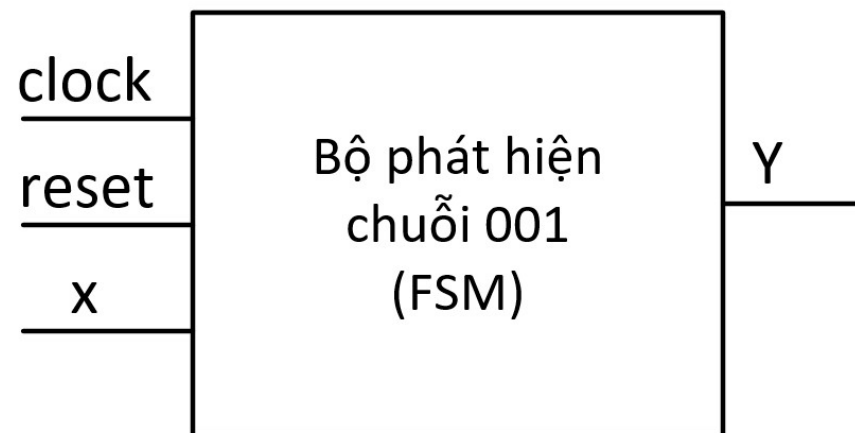
Kết quả mô phỏng



VD 2:

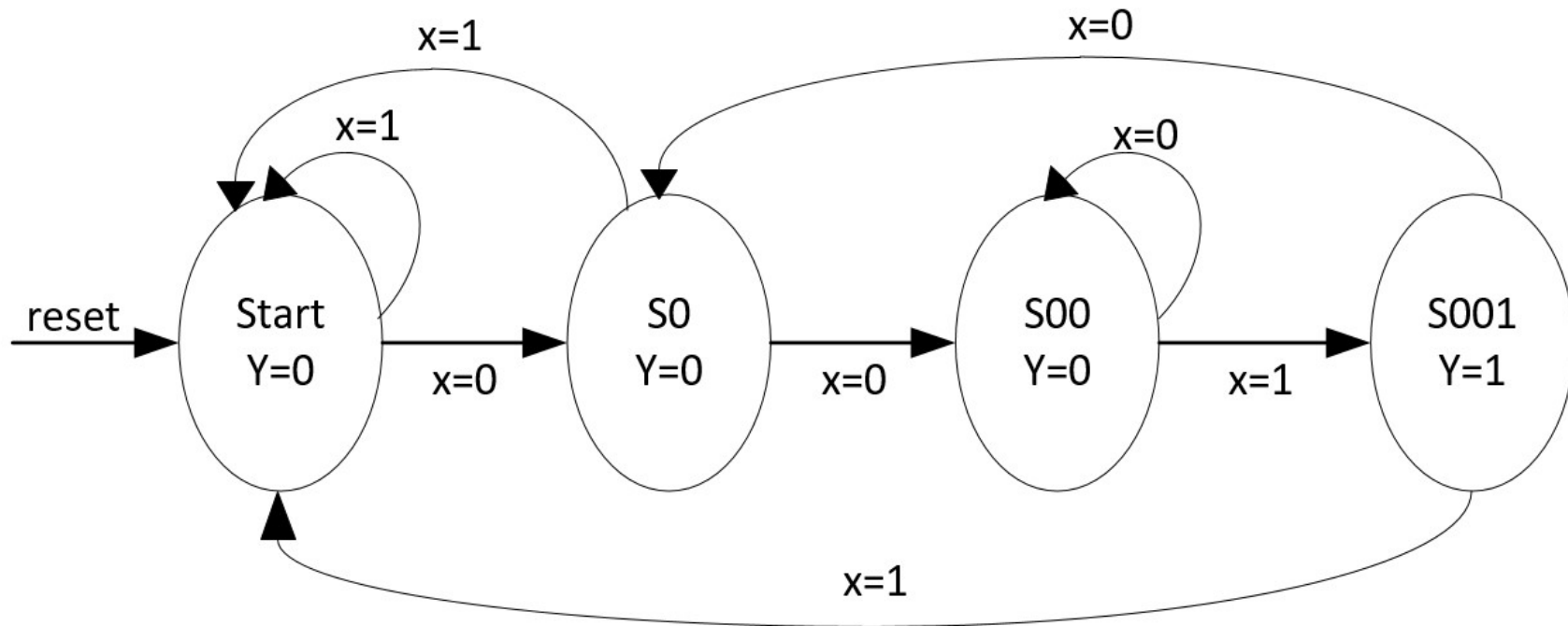
Mô tả FSM có chức năng phát hiện chuỗi 001 từ ngõ vào, rs đồng bộ tích cực thấp, ck tác động theo cạnh lên. Khi phát hiện đúng chuỗi 001 ở ngõ vào thì ngõ ra bằng 1

- Vẽ grap trạng thái
- Viết code verilog



Bài giải:

a. vẽ graph trạng thái



x: ngõ vào

y: ngõ ra

rs : reset tích cực mức thấp

Ck: tác động theo cạnh lên

start, s0, s00, s001 : là các trạng thái trong

Khi phát hiện ngõ vào là chuỗi số 001 thì ngõ ra bằng 1



//khai báo module, tham số, input, output

```
//phát hiện chuỗi 100 ở ngõ vào  
module fsm1(ck,rs,x,y);  
parameter start = 2'b00;  
parameter s0 = 2'b01;  
parameter s00 = 2'b10;  
parameter s001 = 2'b11;  
input ck, rs,x;  
output reg y;  
reg [1:0] next_state, current_state;
```



// mạch tổ hợp xác định trạng thái kế tiếp

```
always @ (*)
begin
  case (current_state)
  start:
    begin
      if (!x) next_state = s0;
      else next_state = current_state;
    end
  s0:
    begin
      if (!x) next_state = s00;
      else next_state = start;
    end
  s00:
    begin
      if (x) next_state = s001;
      else next_state = current_state;
    end
  end
```

```
s001:
  begin
    if (x) next_state = start;
    else next_state = s0;
  end
  default: next_state = start;
endcase
End
```



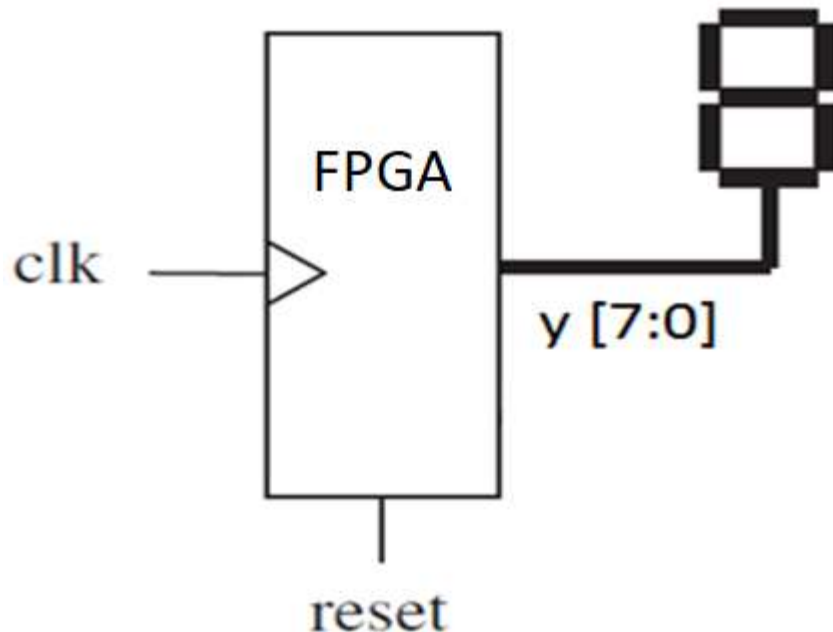
// thanh ghi trạng thái trong và ngõ ra của FF

```
always @(posedge ck)
begin
    if (!rs) current_state <= start;
        else current_state <= next_state;
end
//mạch xác định trạng thái của ngõ ra
always @ (*)
begin
    if (current_state == s001) y = 1'b1;
        else y = 1'b0;
end
endmodule
```



6. Lập trình mô tả mạch ứng dụng

Ví dụ: lập trình điều khiển led bảy đoạn hiển thị số đếm lên $0 \rightarrow 9$, led 7 SEG loại cathode chung, chân rs không đồng bộ tích cực mức cao, thời gian hiển thị 1 trạng thái là 1s.



Gợi ý:

- Viết code đếm nhị phân 0-9
- Viết code giải mã nhị phân 0-9 qua mã led 7SEG

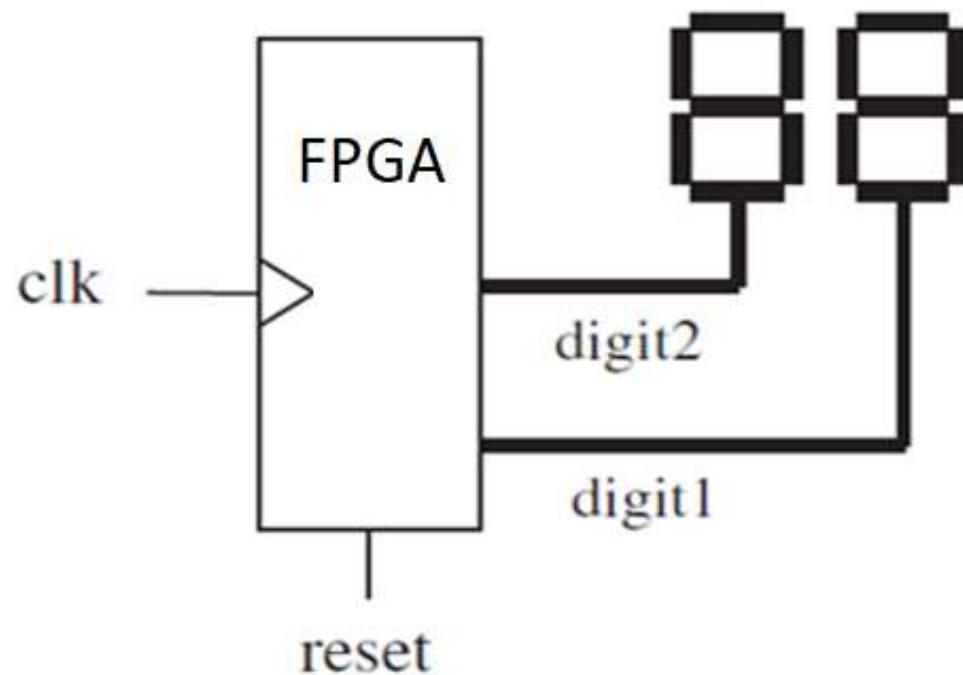


```
//mach dem 0-9, giai ma led 7 seg
module counter_giaima (ck,rs,led);
input ck, rs;
output [7:0]led;
reg [7:0]led;
reg [3:0]q;
always @ (posedge ck )
begin
    if (rs == 1) q <= 4'b 0000;
    else
    begin
        q <= q + 1;
        if (q == 4'b1001)
            q <= 4'b0000;
    end
end
```

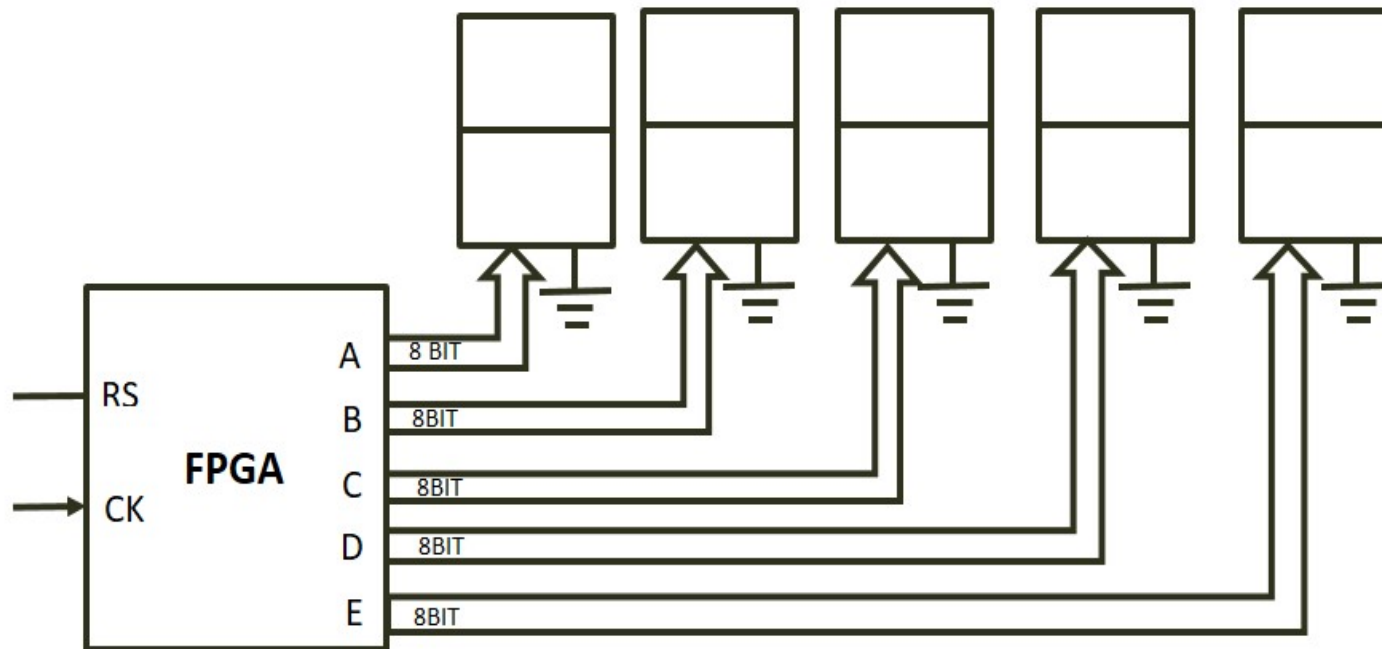
```
// giai ma led 7 seg
always @ (posedge ck)
begin
    case (q)
        4'b0000 : led = 8'HFC;
        4'b0001 : led = 8'H60;
        4'b0010 : led = 8'HDA;
        4'b0011 : led = 8'HF2;
        4'b0100 : led = 8'H66;
        4'b0101 : led = 8'HB6;
        4'b0110 : led = 8'HBE;
        4'b0111 : led = 8'HE0;
        4'b1000 : led = 8'HFE;
        4'b1001 : led = 8'HF6;
    endcase
end
endmodule
```



BT1 : lập trình điều khiển led bảy đoạn hiển thị số đếm lên 00→24, led 7 SEG loại cathode chung, chân rs không đồng bộ tích cực mức cao, thời gian hiển thị 1 trạng thái là 1s



BT2: Viết code verilog lập trình led bảy đoạn hiển thị chữ HELLO. Khi RS tích cực thì led tắt hết. LED bảy đoạn loại cathode chung.



BT4 : Viết code Verilog lập trình điều khiển 10 led đơn sáng dần từ hai biên vào. Khi RS tích cực thì led tắt hết. LED bảy đoạn loại cathode chung. Thời gian tồn tại một trạng thái là 500ms





**Thank you for
listening**

